

IRELE

Image and Video Technology

JPEG compression Project

Author:

Arico Amaury

Professors:

Saeed Mahmoudpour , Colas Schretter

Academic year:

2025 - 2026

1	Introduction	1
2	JPEG compression implementation	2
2.1	Information exploration with ImageJ	2
2.2	Cosine function and quantitative criteria	4
2.3	Discrete cosine transform	5
2.4	2D discrete cosine transform	7
2.5	Data encoding	10
2.6	Entropy and binary encoding	11
2.7	Compression improvements	14
2.7.1	DC delta encoding improvement	14
2.7.2	AC Run-Length encoding improvement	15
3	Conclusion	17

In today's world, the population increasingly relies on data sharing for many daily activities such as communication with relatives, access to information, and media consumption. Data compression has therefore become not only an advantage for transmission speed, but a necessity in modern technologies, where the volume of data to be stored is growing exponentially.

In the following chapters, we present the mathematical foundations of JPEG compression, including the quantization process and the compression strategy underlying this standard. Each step of the algorithm will be discussed and analyzed in order to understand how redundant information is identified and exploited for efficient data compression.

More specifically, the project will be centered around the analysis and compression of a parrot raw image of 256x256 pixels following a methodology similar to the JPEG standard, and will be decomposed into 7 subsections. As foretaste, the first section investigates on the redundant information present in the image using the visualization software **ImageJ**. Subsequently, the implementation phase begins with the development of **basic functions**, such as matrix manipulation and raw image saving, which are required for the remainder of the project. The upcoming steps address successively the implementation of the **DCT coefficients**, the computation of DCT coefficients for **two-dimensional data**, the **quantization** process and finally the **data encoding** to improve the efficiency of the data compression.

JPEG compression implementation

2.1 Information exploration with ImageJ

The initial phase of the project consists of exploring the information contains in the original image and visualize the differences with the JPEG compressed version. The software **ImageJ** with its wide range of functionalities help us in this research. A logical starting point would be to look at the distribution of the pixels value of the original image - Figure 2.1. One can observe that the distribution is spread over the majority of the value range (from 15 to 255) with some concentrated areas. The pixels value distribution for the JPEG image is shown in the figure 2.2. We can already highlight a loss of information due to the quantization, easily visible in the concentrated range between 60 and 100.

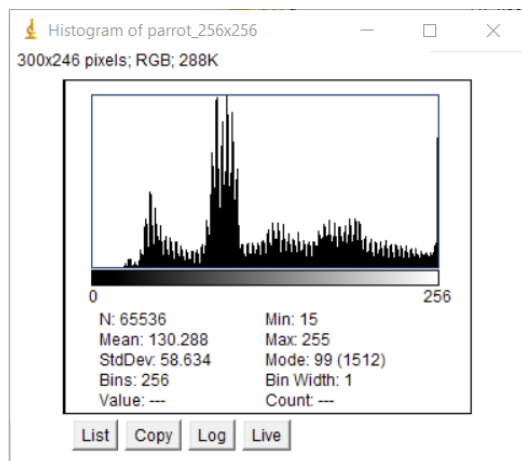


Figure 2.1: Pixels distribution - Original image.

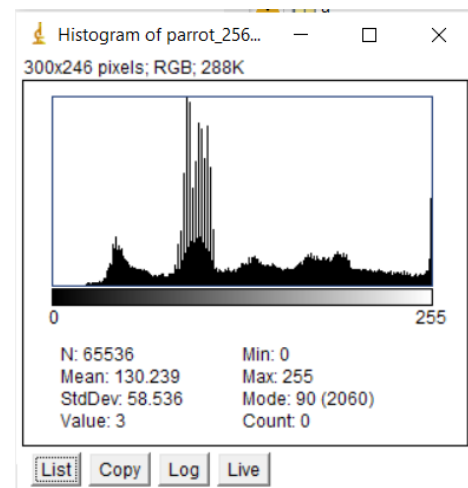


Figure 2.2: Pixels distribution - JPEG image.

This loss of information can be visualize in details by substracting the compressed image from the original image and plot the histogram of the resulting differences - Figure 2.3. The histogram exhibits a laplacian distribution with a 0 mean value and a standard deviation of 4.65. It indicates that the compression does not introduce a biais or alter the luminance of the image, and that the errors introduced by quantization remain limited in amplitude. As a result, no noticeable differences are visible to the naked eye, which is

expalined by the preservation of the low frequency components with negligible quantization error.

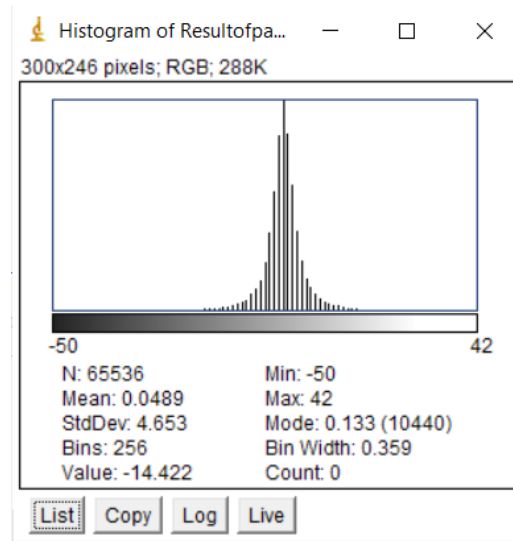


Figure 2.3: Pixels distribution - Original - JPEG substraction.

These high frequency components partially lost during compression can be highlithed by applying a double Gaussian blur operation on the original image and then substract it with a single-blurred image. The result is shown on the figure 2.4. It can be indeed proved that blurring operation acts as a low pass filter (a double blurred operation corresponds to a low pass filter with a cut-off frequency twice smaller than the singe-blurred operation). By substracting the two images, the operation becomes a band-pass filter trapping the frequency components between the two cut-off frequenies.



Figure 2.4: High frequency components enhancement.

Finally, the improvement of the image signal-to-noise ratio obtained by applying a Gaussian blur to a noisy image is investigated. To this end, a white Gaussian white noise image is generated with a standard deviation chosen to match the distortion energy introduced by the blurring operation in the previous study ($\sigma = 8.71$). This noise realization is then added to the original parrot image.

The noisy image is at some restored using Gaussian blur filters with varying standard deviations, and the

mean squared error (MSE) is evaluated for each case. The results show that the MSE reaches a minimum for a blur standard deviation of approximately 0.55. At this value, the reduction of noise energy achieved by the filtering operation is optimally balanced by the distortion introduced by the loss of high frequency image details ($MSE = 39$).

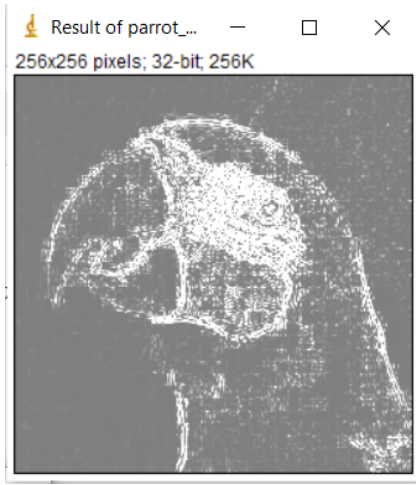


Figure 2.5: Blur filtering errors - Min squared error.

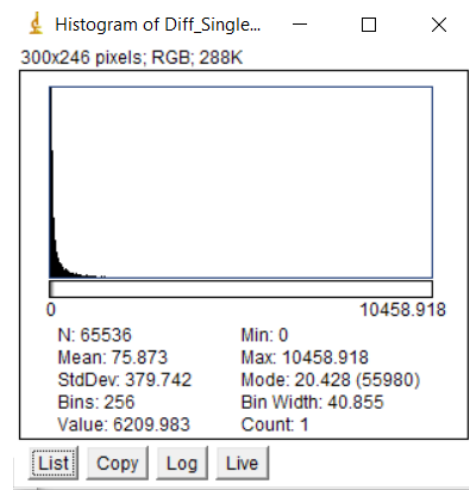


Figure 2.6: Blur filtering errors - Histogram.

2.2 Cosine function and quantitative criteria

The second part of the project is centred around the implementation of basic functions which will be reused in the remainder of the JPEG compression algorithm. The functions cover saving - loading raw data, computing MSE - Peak signal-to-noise ratio (PSNR) and generating a cosine function.

$$I(x, y) = \frac{1}{2} + \frac{1}{2} \cos(x \frac{\pi}{32}) \cos(y \frac{\pi}{64}) \quad (2.1)$$

The fusion between the cosine function and the parrot image can be found in the figure 2.8.

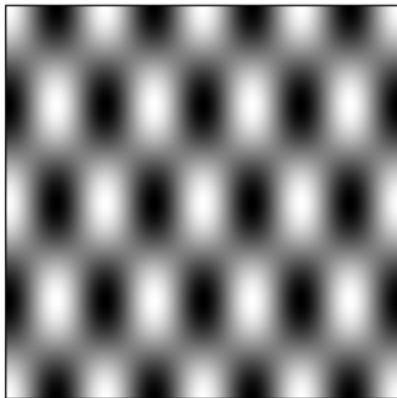


Figure 2.7: Cosine function generation.



Figure 2.8: Fusion of cosine function and original image.

The MSE, PSNR and the maximum pixel value are shown on the figure 2.9 for respectively the **blurred**

parrot, **noisy parrot** and the **sharpened parrot**. One can observe that the noisy and filtered parrot contains the same error energy which is aligned with the image generation methodology done in the first section *Information exploration with ImageJ*. The parrot image sharpened by enhancing the high frequency components exhibits much reduced MSE which was expected (reintroduction of the HF components filtered during the blur process). From a personal perspective, the best visual is achieved without blurring where the noise exacerbates the high frequency transitions providing a texture visual to the image.

```
MSE of blurred parrot is 76.9967
Max of blurred parrot is 255
PSNR of blur parrot is 29.2661

MSE of noisy parrot is 76.2025
Max of noisy parrot is 284.089
PSNR of noisy parrot is 30.2494

MSE of sharpened parrot is 40.6913
Max of sharpened parrot is 261.547
PSNR of sharpened parrot is 32.256
```

Figure 2.9: MSE, maximum pixel value and PSNR for blurred, noisy and sharpened parrot.

2.3 Discrete cosine transform

The subsequent step addresses the computation of the discrete cosine transform coefficient of the analyzed image. These coefficients are obtained by projecting the image onto a set of orthogonal cosine basis functions, which is mathematically equivalent to computing dot products between the image pixel values and the corresponding basis vectors- 2.2.

$$A[k, x] = \cos(k \frac{\pi}{256} (x + \frac{1}{2})), \quad \text{for } k = 0, \dots, 255 \quad \text{and} \quad x = 0, \dots, 255 \quad (2.2)$$

The vectors are normalized to unit energy prior the DCT coefficient computation. The Euclidian norm is calculated by the squared root of the inner product of the basis vectors. The orthonormal basis vectors result from the division of the vectors by its respective Euclidian norms. Mathematically, it boils down to the following equations :

$$E_{\text{basis}} = \mathbf{b}^T \mathbf{b} \quad (2.3)$$

$$\|\mathbf{b}\| = \sqrt{\mathbf{b}^T \mathbf{b}} \quad (2.4)$$

$$\mathbf{b}_{\text{norm}} = \frac{\mathbf{b}}{\|\mathbf{b}\|}, \quad \text{s.t.} \quad \mathbf{b}_{\text{norm}}^T \mathbf{b}_{\text{norm}} = 1 \quad (2.5)$$

These equations are implemented in the functions *energy* and *normalization* in the **Session 2** code which can be found in the zip file attached to the report. The orthonormality of the constructed basis vec-

tors can be easily proved by computing the matrix multiplication AA^T , A being the matrix containing the normalized basis vectors in its rows, and verify that the multiplication returns the identity matrix I - Figure 2.10.

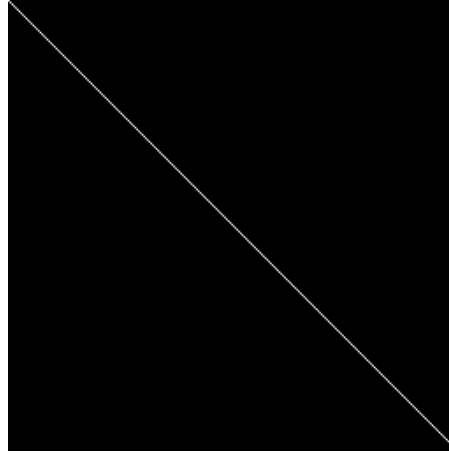


Figure 2.10: Basis vectors orthonormality verification.

The result of the DCT computation on the first 256 pixels of the "cosine" image (generated in the section 2 - *Cosine function and quantitative criteria*) is shown in the figure 2.11. Additionally, a white noise realisation and the parrot image DCT coefficient are also plotted.

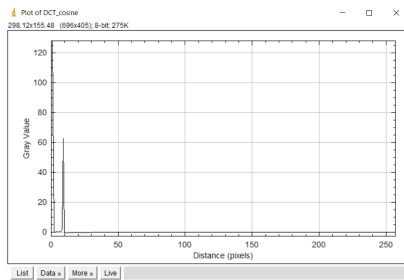


Figure 2.11: DCT coefficients (1D) - cosine image.

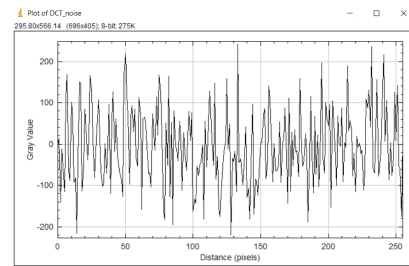


Figure 2.12: DCT coefficients (1D) - noisy realisation.

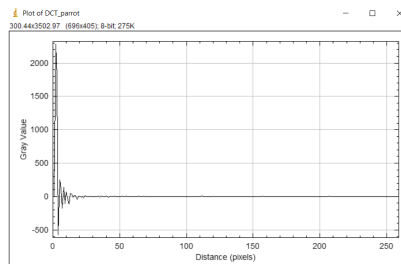


Figure 2.13: DCT coefficients (1D) - parrot image.

It can be observed that, for the cosine image, the DCT coefficients are concentrated around the bin 0 (DC value) and the bin 8 which is consistent with the mathematical proof shown in 2.11.

$$I(x, y) = \frac{1}{2} + \frac{1}{2} \cos(x \frac{\pi}{32}) \cos(y \frac{\pi}{64}) \quad (2.6)$$

$$I(x, 1) = \frac{1}{2} + \frac{1}{2} \cos(x \frac{\pi}{32}) \cos(\frac{\pi}{64}) \quad (2.7)$$

$$\text{considering } \cos x \cos y = \frac{\cos(x+y) + \cos(x-y)}{2} \quad (2.8)$$

$$I(x, 1) = \frac{1}{2} + \frac{\cos(x \frac{\pi}{32} + \frac{\pi}{64}) + \cos(x \frac{\pi}{32} - \frac{\pi}{64})}{2} \quad (2.9)$$

$$\text{knowing that for DCT-II, } \omega = \frac{\pi k}{256} = \frac{\pi}{32} \quad (2.10)$$

$$\mathbf{bin}_{\text{excited}} = \mathbf{k} = 8, \quad (\text{with } k = 0 \text{ for the DC component}) \quad (2.11)$$

The white noise is, as expected, exciting the whole range of frequencies while the majority of the parrot image energy is localized in the low frequency band. By setting a significant threshold of 100 on the DCT coefficient amplitudes, setting all smaller coefficients to zero, an image with a PSNR of 29.83dB is reconstructed. This value is similar to one achieved by the blurred image discussed during the first section, meaning that the details of the image is partially lost while the overall image structure remains recognizable. It is important to note that JPEG compression does not quantize DCT coefficients by simply discarding low-amplitude values as done here. Instead, the standard applies frequency-dependent quantization, where each coefficient is divided by a value from a quantization table and then rounded to the nearest integer.

2.4 2D discrete cosine transform

From this point, the remainder of the project will make use of the two-dimensional Discrete Cosine Transform. The mathematical foundation behind the 2D DCT can be described as a two-times image projection on the same orthonormal basis vectors. The first projection is computed as in the previous section, where the second projection is applied after transposing the result of the first projection. It can be expressed analytically as follow :

$$DCT_{2D} = (A(AX)^T)^T, \quad \text{Where X is the original image} \quad (2.12)$$

$$\text{equivalently } DCT_{2D} = AXA^T \quad (2.13)$$

$$IDCT_{2D} = A^T Y A, \quad \text{Where Y is DCT coefficients} \quad (2.14)$$

The 2DCT of the raw image and the 2DCT of the same image segmented into block of 8x8 pixels are shown in figures 2.14 and 2.15.

Data can be additionally compressed by applying a quantization strategy. The JPEG quantization was briefly explained in the end of the third section. The following provides a more detailed explanation on the

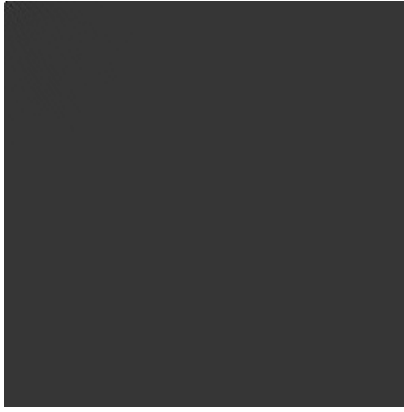


Figure 2.14: 2DCT coefficients - no segmentation.

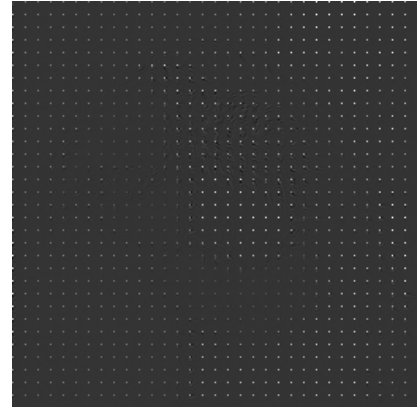


Figure 2.15: 2DCT coefficients - 8x8 blocks.

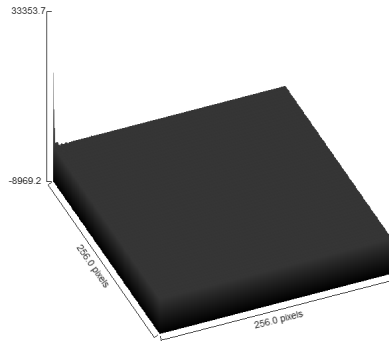


Figure 2.16: 2DCT coefficients - Surface plot.

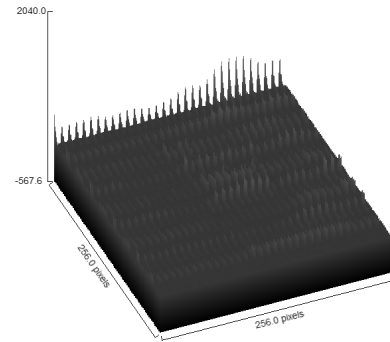


Figure 2.17: 2DCT coefficients - 8x8 blocks - Surface plot.

mathematical approach and the impact on reconstructed image visual. As explained previously, the standard is built on a frequency-dependent quantization, meaning that high frequency coefficients are subject to strong quantization opposingly to the lower frequency coefficients exposed to a finer one 2.1.

$$Q = \begin{bmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{bmatrix}$$

Table 2.1: JPEG quantization table (quality factor 50).

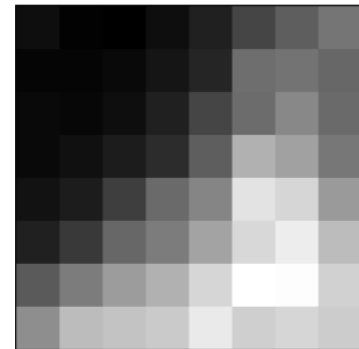


Table 2.2: JPEG 50 Quantization table.

This gives a gradual information loss suitable to the human visual system, much sensitive to low frequency information. In the contrary, the threshold-based quantization is a binary operation, where information is either fully lost or preserved potentially leading to abrupt visual effect. That said, **for small quantization**, the parrot image with thresholding leads to much pleasant visual than with the JPEG quantization for the same PSNR (41dB) due to the JPEG-inherent ringing waves at high frequency pixels transitions -

figures 2.18 and 2.19.



Figure 2.18: JPEG 50 quantization - **psnr** = 41dB.



Figure 2.19: Thresholding quantization - **psnr** = 41dB.

For the completeness, the comparison of both methods for a much significant quantization (still same PSNR - 24dB) are shown in figures 2.20 and 2.21. One can clearly see the low frequency preservation by the JPEG quantization.



Figure 2.20: JPEG quantization - **psnr** = 24dB.

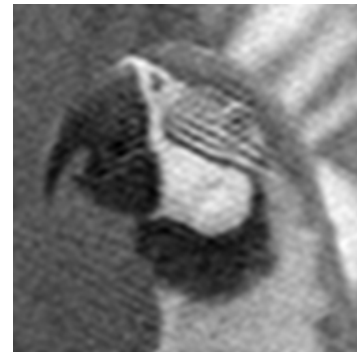


Figure 2.21: Thresholding quantization - **psnr** = 24dB.

Finally, the figure 2.23 plots the residuals of the difference between the original image and the JPEG 50 reconstructed image, showing, as expected, a laplacian centered in zero - in line with what we discussed in the first section of this report *Information exploration with ImageJ*.

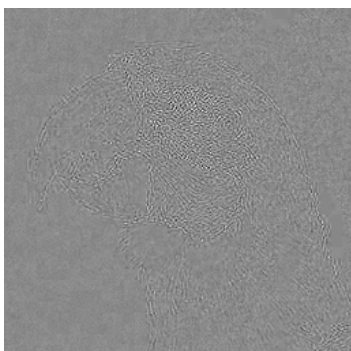


Figure 2.22: JPEG - original residuals.

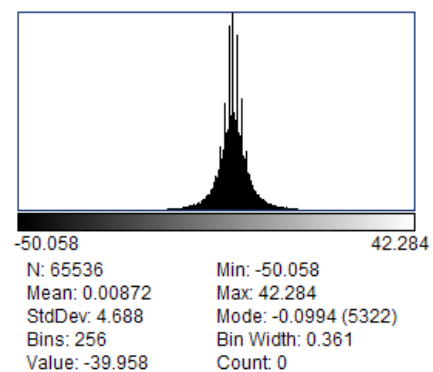


Figure 2.23: Residuals - histogram.

2.5 Data encoding

Once quantization has been applied, the final step consists in encoding the resulting data and computing the compression rate achieved by the proposed strategy. The encoding stage plays a crucial role, as it is where the compression rate can be significantly improved. To illustrate this point, Figure 2.24 presents a histogram of coefficient occurrences, showing the frequency with which identical DCT coefficients appear after quantization.

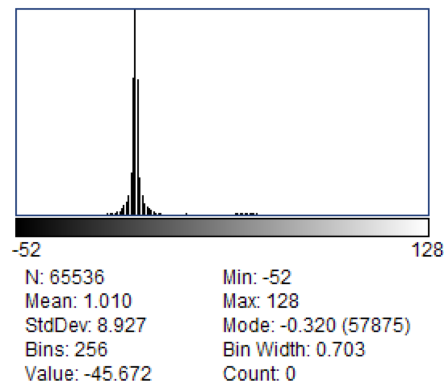


Figure 2.24: Number of occurrences - Quantized DCTs.

The histogram clearly shows a strong concentration of coefficient values around 0, highlighting the large number of high frequency DCT coefficients that are set to zero after JPEG quantization (57875). This property can be effectively exploited to reduce the amount of data that must be transmitted to the receiver. It therefore remains to define an encoding strategy capable of taking advantage of this coefficient concentration.

The encoding process begins by separating the DC and AC components. This separation is necessary because the statistical distributions of DC and AC coefficients differ significantly. DC coefficients exhibit strong correlations between neighboring blocks, whereas the AC coefficients follow a sparse distribution centered around zero after quantization. These distinct statistical properties lead to two different encoding strategies, respectively the delta encoding and the run-length encoding.

The delta encoding is based on a simple mathematical subtraction of the previous block DC coefficient with one at its neighborhood. For the initial implementation, it was chosen to apply this subtraction sequentially from the first block stored in the memory to the last one. The result of the delta-encoding is shown in the table 2.3 for the first 33 elements.

1	2	3	4	5	6	7	8	9	10	11	.	.	.	.	.	30	31	32	33
51	2	0	0	0	0	-4	-5	0	5	4	.	.	.	.	.	-6	2	-6	-36

Table 2.3: Quantized DCT coefficients encoded in zigzag order.

Two remarks can be made. First, the subtraction reduces significantly the magnitude of DC coefficients, making them much more efficient for the binary encoding. Second, some high DC values appear at indices

$i * 32 + 1$. This behaviour is directly related to the DC block encoding order. Indeed, every 32 elements, the delta encoding process switches to a new row of blocks, failing to exploit the spatial correlation between neighboring blocks. The last section of the report *Compression improvements* will investigate in greater details how to improve the DC encoding (and the AC encoding).



Figure 2.25: 8x8 block DC coefficients.

As mentioned previously, the AC encoding rests on the run-length encoding (RLE) process. The strategy works as follow : for each block, the algorithm scans the AC coefficients sequentially. When a zero is encountered, a counter is incremented. Upon reaching a non-zero value, the counter is memorized, the non-zero value is stored with the counter and the couple (zero-counter, non-zero value) is saved as RLE symbol. This process is iterated over all the block elements - for each block.

$$\mathbf{RLE}_{\text{symbol}} = (\mathbf{counter}_{\text{zero}}; \mathbf{val}_{\text{non-zero}}) \quad (2.15)$$

The figure 2.26 shows the encoded RLE couples of the 4th block (2 sequences).

```
RLE sequence for the 4th block of 8x8{ zeros = 0, values = [-1] }
RLE sequence for the 4th block of 8x8{ zeros = 62, values = [] }
```

Figure 2.26: RLE sequences - 4th block.

2.6 Entropy and binary encoding

Entropy, as introduced by Shannon, quantifies the average amount of information, or uncertainty, contained in a data source. It represents the minimum average number of bits required to encode the source. In thermodynamics, entropy describes the level of disorder of a system, which conceptually aligns with the definition used in information theory. The mathematical expression of entropy is given in Eq. 2.16 and is directly related to the statistical properties of a random variable.

$$H(X) = - \sum_{i=1}^N p(x_i) \log_2 p(x_i) \quad (2.16)$$

where $p(x_i)$ denotes the probability of the outcome x_i .

Applying entropy computation to the run-length encoded (RLE) symbols provides the theoretical minimum number of bits per symbol required for encoding. Given the set of RLE symbols extracted from the parrot image, the remaining step consists in estimating the probability of each symbol. This is achieved by counting the number of occurrences of each symbol. The figure 2.27 shows the normalized occurrence distribution for all unique generated during the encoding. It is worth to recall that the RLE symbol is composed of **two** distinct variables - a counter of , or zero run-length and a non-zero coefficient value.

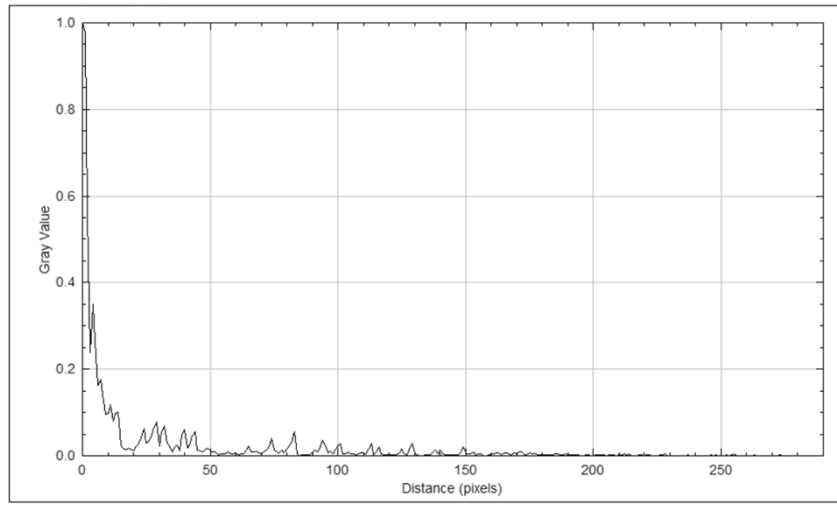


Figure 2.27: Normalized occurrences for each 289 unique RLE symbol.

The plot, 2.27, is normalized by the maximum number of occurrences - 1057 occurrences. The most frequently encountered RLE symbol during encoding corresponds to the couple (**counter**_{zero}; **val**_{non-zero}) = (0, -1).

After normalization of the occurrence counts by the total number of symbols, ensuring that the probabilities sum to 1, the entropy (**H**_{AC}) for the current encoding is found to be 5.466 **bits per symbol**. The same process has been done for the DC coefficients after delta encoding. It results to an entropy (**H**_{DC}) of 6.377 **bits per symbol**. The weighted entropy is computed in the equation 2.17.

$$\mathbf{H}_{\text{weighted}} = \frac{N_{DC} H_{DC} + N_{AC} H_{AC}}{N_{DC} + N_{AC}} = 5.5879 \text{ bits per symbol} \quad (2.17)$$

with (**N**_{DC}) and (**N**_{AC}) denoted as number of DC symbols and AC symbols.

The last part of this section will focus on the binary encoding of the DC and AC encoded symbols. In that context, the Exponential Golomb (EG) is employed, as it is particularly well suited for representing signed integers as those contained in the AC and DC symbols with a laplacian distribution centered around 0 - the

AC non-zero coefficients distribution is shown in figure 2.28.

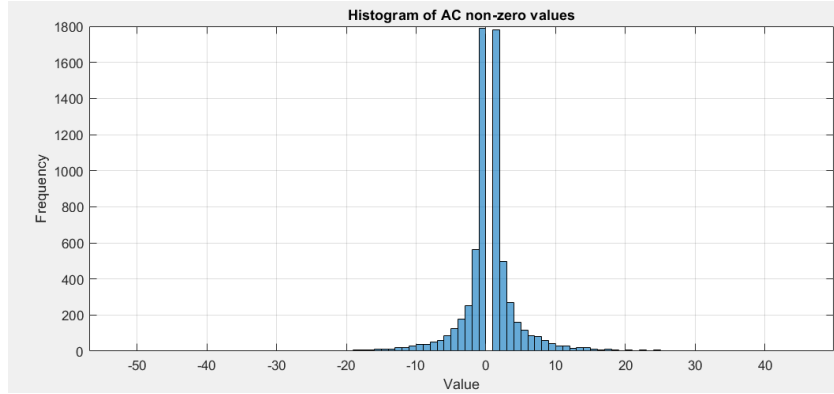


Figure 2.28: AC non zero values distribution after quantization.

Unlike Huffman coding, the Exponential Golomb is not using the frequency appearance of the symbols to organize the symbol coding making it less optimal. Nevertheless, EG coding provides a fast and simple coding computation, and formulated as expressed in the equation 2.18.

$$\mathbf{u} = \begin{cases} 2n, & \text{if } n \geq 0, \\ -2n - 1, & \text{if } n < 0. \end{cases} \quad (2.18)$$

The mathematical expression in 2.18 provides a signed-to-unsigned mapping shown on the table 2.4.

Original value n	Mapped value u
0	0
1	2
2	4
-1	1
-2	3
-3	5

Table 2.4: Signed to unsigned mapping.

Finally, the Exponential Golomb coding is composed of a prefix of number of zeros informing about the number of bits used to code the mapped value and a suffix expressing the same mapped value in binary number incremented to 1. for clarity, an example is provided below.

Example

As $n < 0$, the mapped value is:

$$u = -2(-5) - 1 = 9. \quad (2.19)$$

The Exponential-Golomb code number is then computed as:

$$\text{code} = u + 1 = 10. \quad (2.20)$$

The binary representation of the code number is:

$$\mathbf{binary} = 1010. \quad (2.21)$$

The number of leading zeros is given by:

$$k = \text{number of bits} - 1 = 3. \quad (2.22)$$

The final Exponential-Golomb codeword is constructed by concatenating the prefix of k zeros and the binary representation of the code number using $k + 1$ bits:

$$\underbrace{000}_{\text{prefix}} \underbrace{1010}_{\text{suffix}}. \quad (2.23)$$

Thus, the Exponential-Golomb encoding of the integer $n = -5$ is:

$$\boxed{0001010}. \quad (2.24)$$

Compression rate and bits per symbol

The combination of **zigzag DC delta encoding**, **zigzag AC Run Length encoding** and **Exponential Golomb coding** generated a total of 63984 bits for the parrot image. From the bitstream, the compression rate and the bits per symbol can be computed 2.25.

$$\begin{aligned} \text{Bits per pixel} &= \frac{\text{Total bits}}{\text{Total pixels number}} = \frac{63984}{65536} = 0.9762 \text{ bpp}, \\ \text{Compression} &= \frac{\text{Raw bits per pixel}}{\text{Compressed bits per pixel}} = \frac{8}{0.9762} = 8.1948, \\ \text{Bits per symbol} &= \frac{\text{Total bits}}{\text{Total symbols}} = \frac{63984}{1024 + 6637} = 8.3519 \text{ bps}. \end{aligned} \quad (2.25)$$

One can notice that the bits per symbol are indeed higher than the entropy computed in 2.17. The resulting compressed binary file is still 35 percents bigger than the JPEG format. The final section will implement encoding optimization in order to reach the same level of the JPEG compression.

2.7 Compression improvements

2.7.1 DC delta encoding improvement

The first optimization concerns the DC delta encoding. The section about the data encoding briefly explains the miss of compression due to the row jumping each 32 elements. The core idea of the optimization consists to change the encoding order from a zigzag path to a **serpentine** path - Figure 2.30.

The entropy, compression rate and bits count after optimization are re-computed in 2.5.

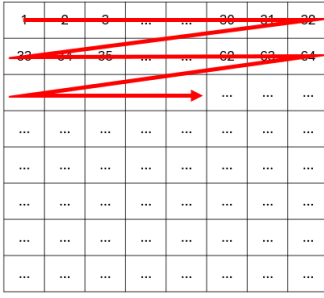


Figure 2.29: Zigzag path.

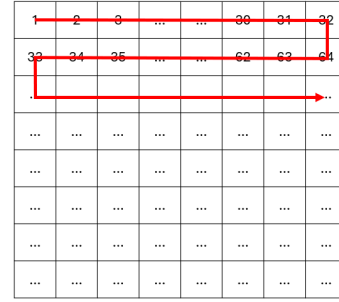


Figure 2.30: Serpentine path

Metric	Before Optimization	After Optimization
Entropy (DC, bits/symbol)	6.37	5.45
Compression Ratio	8.19	9.25
Bits per Pixel (bpp)	0.97	0.86
Bits per Symbol (bps)	8.35	7.35

Table 2.5: Comparison of image coding performance before and after delta encoding optimization.

The total bits number amounts to 56656 bits, which translates to a reduction of **13 percents** from the previous result in the section 2.6 but still **20 percents** higher than JPEG image.

2.7.2 AC Run-Length encoding improvement

The second optimization treats about the Run-Length encoding optimization. In a same way the serpentine path improved the DC encoding, an optimized path can be found for the AC encoding too. The strategy is to encode the non-zero coefficients first and group the zero coefficients afterwards. As the JPEG quantization tends to set the high frequency components to zero, the most optimized index reorganization is the one sorting the coefficients from the smallest euclidian distance from the DC coefficient to the farthest. The path is shown on the figure 2.31.

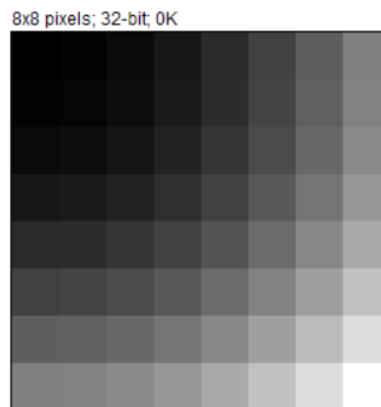


Figure 2.31: Normalized occurrences for each 289 unique RLE symbol.

Equivalently to what was done for the delta encoding, the table 2.6 shows the encoding performance improvement for the AC Run-Length encoding.

The total bits number amounts to 50216 bits, which translates to an additional reduction of **13 percents**

Metric	Before Optimization	After Optimization
Entropy (AC, bits/symbol)	5.46	4.91
Compression Ratio	9.25	10.44
Bits per Pixel (bpp)	0.86	0.76
Bits per Symbol (bps)	7.35	6.55

Table 2.6: Comparison of image coding performance before and after Run-Length encoding optimization.

from the previous optimization but still **6.5 percents** higher than JPEG image.

This report relates the progress achieved over seven weeks of coding but, more importantly, seven weeks of reflection about DCT transform and encoding strategies. Through the implementation from scratch of the Discrete Cosine Transform and the encoding seen in classes, the project helped to anchor the understanding and roles of the discrete frequency analysis of a digital signal processing. Better yet, the use of ImageJ software enabled an intuitive visualization of this frequency analysis which can appear abstract to newcomers in this domain. The report concludes with the image reconstructed after quantization and after applying the improved delta and RLE encoding processes developed during the practice sessions, as shown in Figure 3.1.



Figure 3.1: Reconstructed parrot image.